

講義 21：建立 Z 資料表與 Open SQL 寫入（授課順序：接在講義 25 之後、講義 8a 之前）

對應練習：[ex21](#) | 答案物件：資料表 [ZTR21_STUD](#) + [ZTR21_CLASS](#)、Search Help [ZTR21_CLASSH](#)、Function Group [ZFG_TR21](#)（SM30 維護畫面）+ 程式 [ZR_TR21_ZTABLE](#)

本講用 SE11、SM30 與 Open SQL，把 Z 表「建起來、維護、寫進去」。Maintenance View、Help View 與 JOIN 串表需要先學 JOIN（講義 11），放在[講義 21a](#)（第一階段總整理講義 13 之前）。

本講重點

- DDIC 三層件：Domain → Data Element → 表格欄位，各管什麼
- SE11 建立透明表：鍵欄位、Delivery Class、Technical Settings
- Fields 頁籤的 Key（主鍵）與 Initial Values（能不能是 NULL）
- 金額（[CURR](#)）、數量（[QUAN](#)）欄位要指定 [CUKY](#)（幣別）/ [UNIT](#)（單位）型別的參考欄位
- **Header / Detail** 兩表關聯：外鍵（Foreign Key）與檢查表（Check Table）
- **Search Help**：F4 值清單怎麼來的，跟外鍵的差別
- SM30 維護畫面（Table Maintenance Generator），在畫面上驗證值域、外鍵、F4
- Open SQL 寫入：[INSERT](#) / [UPDATE](#) / [MODIFY](#) / [DELETE](#)
- LUW 與 [COMMIT WORK](#) / [ROLLBACK WORK](#) 觀念

1. DDIC 三層件：欄位是怎麼組成的

講義 6 學過「讀」標準表；講義 25 講過 DDIC 的系統地圖與 Global Type 觀念（何時重用標準型別、何時自建）——本講接著把「自建」這條路完整動手走一次：客製開發常需要自己的表存資料（參數表、log 表、暫存表——本專案的 ZDQM 系列就有）。SE11 的欄位定義是三層疊起來的：

層	管什麼	例
Domain（值域）	技術屬性：型別、長度、小數位、允許值清單（Value Range）、轉換常式	INT4、值域 0 ~ 999
Data Element（資料元素）	語意：欄位標籤（F1 說明、畫面上的欄位名稱，可多語言）	「學生成績」
表格欄位	引用一個 Data Element（或直接用內建型別）	SCORE TYPE ztr21_score

為什麼分三層？**重複利用與一致性**：十張表都有「成績」欄位時，共用同一個 Data Element，標籤、F1 說明、值域全系統一致，改一處全生效。標準表的欄位全是這樣組的——這也是為什麼 [TYPE scarr-carriid](#) 能帶出那麼多語意（講義 6）。

實務折衷：**鍵欄位與有業務意義的欄位用 Data Element**；純技術性欄位（旗標、備註）可直接用內建型別（CHAR、INT4...）省事。課程練習兩種都做。

2. SE11 建立透明表（Transparent Table）

步驟（細節照練習 ex21 的規格做）：

1. SE11 → Database table → 輸入 **ZTR21_STUD** → Create
2. **Delivery Class**：A（應用資料，最常用）。速查：

Class	用途
A	應用資料（主檔/交易資料）——預設選它
C	客戶自訂設定檔（Customizing，會跟著 TR 搬）
L	暫存資料

1. Data Browser/Table View Maint.：選 **Display/Maintenance Allowed**（之後 SE16N 才能看、第 4 節的 SM30 才能維護）
2. **Fields** 頁籤：第一欄一定是 **MANDT**，型別填標準 Data Element **MANDT**（Domain **MANDT**，**CLNT** 長度 3），且勾 **Key**；接著鍵欄位、資料欄位。每張 Z 表都要這樣放：第一個 Key 欄位是 **CLNT** 型別，這張表才是 client 相關表，Open SQL、SE16N、SM30 才會自動只處理登入的 client；漏放或放到後面，表就變成跨 client 共用，別的 client 也看得到你的資料（完整說明見講義 25 §6.3）
3. **Technical Settings**（必填才能啟用）：Data Class **APPL0**（主檔類）、Size Category **0**（預估筆數級距，練習表選最小）
4. Enhancement Category（選單 Extras）：選 **Can be enhanced** 或 **Cannot be enhanced** 皆可（練習表無所謂，正式表依團隊規範）
5. 啟用（Ctrl+F3）——表就真的建在資料庫了，SE16N 可立刻查

命名：客製表 Z 開頭（本課 **ZTR21_STUD**）；欄位名限 16 字元。表建錯欄位型別，上線後要改很痛（要做轉檔），設計階段多想一分鐘。

2.1 Fields 頁籤的 Key 與 Initial Values 勾選框

Key：主鍵（Primary Key）

Fields 頁籤每個欄位最左邊的 **Key** 勾選框，勾起來的欄位合起來就是這張表的主鍵：用來唯一識別一筆資料，表裡不會有兩筆資料的主鍵完全相同（官方文件 **ABENDDIC_DATABASE_TABLES_KEY**）。

怎麼決定哪些欄位當 Key：問自己「什麼組合可以唯一確定一筆資料？」

表	主鍵	意思
ZTR21_STUD （學生）	MANDT + ID	一個學號一筆
ZTR25_SURCHG （加成設定，講義 25）	MANDT + CARRID	一家航空公司一筆設定
ZFI0037 （會計主管，講義 13）	MANDT + BUKRS + INAUGURATION	同一家公司、同一個生效日一筆；換主管就新增一筆新的生效日
MAKT （物料說明）	MANDT + MATNR + SPRAS	一個物料、一種語言一筆

規則：

- 至少要有一個 Key 欄位；第一個一定是 client 欄位 **MANDT**。

- Key 欄位必須連續排在最前面，中間不能夾非 Key 欄位。
- 最多 16 個 Key 欄位，總長度不超過 900 bytes；超過 120 bytes 時不能當 Lock Object 的主表（講義 27）。
- FLTP、STRING、RAWSTRING、LCHR、LRAW 這些型別不能當 Key。
- Key 欄位一定不能是 NULL，所以 Initial Values 會自動勾選（見下方）。

Key 對程式的影響：

- INSERT 一筆主鍵已存在的資料會失敗，sy-subrc = 4（第 5 節）。
- MODIFY 靠主鍵判斷：主鍵存在就更新、不存在就新增。
- SELECT SINGLE 在 WHERE 裡給齊所有 Key 欄位，才保證讀到的就是那一筆。
- 系統會依主鍵自動建主索引（Primary Index），WHERE 條件有給 Key 欄位時讀取很快。

主鍵要在建表時想清楚。表裡已經有資料之後再改 Key，要做資料轉換，很麻煩。

Initial Values：資料庫欄位能不能是 NULL（DDL 的 not null）

每個欄位的 Key 勾選框右邊還有一個 Initial Values（畫面欄寬窄時顯示成 Init...）勾選框。用 DDL 寫時就是欄位後面的 not null，例如 key matnr : matnr not null;。

它決定這個欄位在資料庫裡能不能是 NULL（官方文件 ABENDDIC_DATABASE_TABLES_INIT）：

勾選	資料庫欄位	沒給值的資料列會是
有勾	NOT NULL	型別的初始值：字元是空白、數字是 0、日期是 00000000
沒勾	看資料庫與情況，可能是 NULL	可能是 NULL（沒有值）

NULL 是什麼：資料庫裡「完全沒有值」的狀態，跟空白、0 都不一樣。ABAP 沒有 NULL 這種值，讀進程式後會變成初始值；但在 WHERE 條件裡，NULL 不等於空白：WHERE klasse = space 找不到 NULL 的資料列，要寫 WHERE klasse IS NULL 才找得到。

什麼時候要注意：

- **Key 欄位：**勾了 Key，系統就會自動把 Initial Values 也勾起來，不用自己勾。
- **建一張新表：**大部分資料庫平台建表時，所有欄位本來就會是 NOT NULL，勾不勾差別不大。
- **在已經有資料的表加新欄位：**這才是關鍵。沒勾的話，舊資料列的新欄位在某些資料庫會是 NULL，之後用 WHERE 新欄位 = space 會漏掉這些舊資料；有勾的話，系統會把舊資料列填成初始值。但表很大時，填初始值會花很久，官方建議只在必要時、或資料量不大的表才勾。
- 以下型別不能勾：LCHR、LRAW，以及長度 70 以上的 NUMC、RAW。

本課程的練習表照 ex21 的規格，Key 欄位勾選 Key + Initial Values 即可。

2.2 金額欄位要配幣別、數量欄位要配單位

表格裡如果有金額或數量欄位，只設定欄位型別還不夠，一定要再指定一個「參考欄位」，否則表無法啟用（報 specify reference table AND reference field）：

欄位型別（Domain 的 DDIC 型別）	必須參考的欄位型別
CURR（金額）	CUKY（幣別代碼，如 TWD、JPY）

欄位型別 (Domain 的 DDIC 型別) 必須參考的欄位型別

QUAN (數量)

UNIT (單位 , 如 KG 、 PC)

參考欄位的用途 : 告訴系統這個數字是什麼幣別 / 單位 , 並由它決定小數位 (官方文件

ABENDDIC_CURRENCY_FIELD 、 ABENDDIC_QUANTITY_FIELD) 。

金額 (CURR + CUKY)

- 各幣別有幾位小數 , 設定在表 TCURX (欄位 CURRDEC , 交易碼 OB08 相關設定) ; 沒列在 TCURX 的幣別一律 2 位。
- 最小單位 : 每種幣別能細分到的最小金額 , 由它的小數位數決定。USD 是 2 位 , 最小單位是 1 美分 (0.01 美元) ; JPY 是 0 位 , 最小單位就是 1 日圓 ; KWD 是 3 位 , 最小單位是 1 fils (0.001 第納爾) 。
- SAP 存金額的方式 : CURR 欄位的小數位在 DDIC 定義時就固定了 (通常 2 位) , 不會因為每筆資料的幣別不同而改變。存的時候分兩步 :
 - 先算這筆金額是「幾個最小單位」 : 10,000 日圓 = 10000 個日圓。
 - 把這串數字 10000 直接放進 2 位小數的欄位 , 最後兩位被當成小數 , 所以存成 100.00 。

實際金額	最小單位個數	資料庫存的值 (2 位小數欄位)
100.00 美元	10000 美分	100.00
10,000 日圓	10000 日圓	100.00
10.000 第納爾	10000 fils	100.00

- 所以資料庫裡的 100.00 本身看不出是多少錢 , 一定要搭配幣別欄位才知道。
- 顯示時依參考欄位的幣別放小數點。本系統實測 (2026-09-30 , TCURX 查證 + WRITE ... CURRENCY 實跑) , 同一個值 100.00 :

幣別	TCURX 小數位	顯示
USD	不在 TCURX , 預設 2	100.00
TWD	0 (本系統設定)	10,000
JPY	0	10,000
KWD	3	10.000

數量 (QUAN + UNIT)

- 各單位有幾位小數 , 設定在表 T006 (欄位 DECAN , 交易碼 CUNI) 。
- 顯示時依單位的小數位 , 把多出來、而且是 0 的小數去掉 ; 不是 0 的照樣顯示 , 不會四捨五入。本系統 KG 的 DECAN 是 0 : 12.000 顯示成 12 , 12.500 仍顯示 12.500 。

什麼地方會自動用參考欄位 , 什麼地方不會

場合	會不會自動依幣別 / 單位決定小數位
SM30 、 Dynpro 畫面	會。系統找到參考欄位 , 依它的幣別 / 單位顯示 , 也依它檢查輸入

場合	會不會自動依幣別 / 單位決定小數位
程式 WRITE	不會，要自己加 WRITE ... CURRENCY 幣別 或 WRITE ... UNIT 單位 (講義 12)
Open SQL 讀寫	不會，就是一般的數字
程式計算	用 DDIC 定義的小數位計算。兩個金額相乘、不同幣別的金額互相運算，結果都不可靠

所以報表程式印金額時一定要加 CURRENCY：日圓的 100.00 不加 CURRENCY 印出來就是 100.00，實際卻是 10,000 元。

參考欄位通常放在同一張表裡。標準表 SBOOK (航班訂位) 就是這樣設計：

金額 / 數量欄位	參考欄位
LUGGWEIGHT (行李重量，QUAN)	WUNIT (重量單位，UNIT)
FORCURAM (外幣金額，CURR)	FORCURKEY (外幣幣別，CUKY)
LOCCURAM (本幣金額，CURR)	LOCCURKEY (本幣幣別，CUKY)

一張表可以有多個金額欄位，各自指向不同的幣別欄位 (如上面的外幣、本幣)。

SE11 設定方式：在表格的 **Currency/Quantity Fields** 頁籤，每個金額 (或數量) 欄位那一列有兩格要填：

欄位	意思
Reference table (參考表)	幣別 / 單位欄位在哪一張表 (或結構) 裡
Ref. field (參考欄位)	那張表裡的哪一個欄位；型別必須是 CUKY (金額) 或 UNIT (數量)

參考表可以是別張表。標準表 MARC (物料的工廠資料) 就是例子 (SE11 → MARC → Currency/Quantity Fields 頁籤)：

MARC 的欄位	型別	Reference table	Ref. field	為什麼參考別張表
MINBE (再訂購點)、 EISBE (安全庫存)	QUAN	MARA	MEINS	數量的單位是物料的基本單位，只在物料主檔 MARA 記一次，各工廠共用，MARC 不再重複存一份
LOSFX (與批量無關的成本)	CURR	T001	WAERS	金額的幣別是公司代碼的幣別，定義在公司代碼主檔 T001

參考別張表時要注意：參考表只告訴系統「幣別 / 單位在哪個欄位」，值不會自己出現。畫面顯示 MARC-MINBE 時，系統到程式的全域資料裡找 MARA-MEINS 這個欄位拿單位；如果程式沒把那筆物料的 MARA 讀進來，就找不到單位，數量會照一般數字顯示 (官方文件 ABENDDIC_QUANTITY_FIELD)。程式 WRITE 時也一樣，要自己讀出單位再寫 WRITE ... UNIT。

自建表時，最單純的做法是把幣別 / 單位欄位放在同一張表 (如上面的 SBOOK)，讀一筆資料就同時拿到金額和幣別，不用另外去讀別張表。

用 DDL 寫的話，是在金額 / 數量欄位前面加 annotation (以下節錄自 **SB00K**)：

```
@Semantics.quantity.unitOfMeasure : 'sbook.wunit'  
luggweight : s_lugweigh not null;  
wunit      : s_weiunit not null;  
  
@Semantics.amount.currencyCode : 'sbook.forcurkey'  
forcuram   : s_f_cur_pr not null;  
forcurkey  : s_curr not null;
```

- 參考欄位本身的型別必須是 **CUKY** / **UNIT**，通常直接重用標準 Data Element (如 **WAERS**、**MEINS**)，不要自建 (講義 25 第 2 節)。
- DDIC 結構 (Structure) 也是同樣規則，講義 15 建 **ZTR15_FLIGHT_REV** 時就遇過。
- 各型別的對照見講義 25 第 1.2 節。

SE11 頁籤名稱依一般 SAP GUI 版本整理，如果你看到的畫面不一樣，請回報，講義會再修正。

3. Header / Detail 關聯：外鍵 (Foreign Key) 與檢查表 (Check Table)

實務上很少有表是孤立的：訂單表要串客戶主檔、明細表要串產品主檔——本質都是「**Header** (主檔，1 那一邊) / **Detail** (明細，多那一邊)」的關聯。本課用「班級 (Header) - 學生 (Detail)」示範，跟講義 6 的 SCARR (航空公司) - SPFLI (航線) 是同一種關係，只是這次自己動手建。

3.1 觀念：Check Table 是什麼

- **檢查表 (Check Table)**：被參考的那張表 (本例 **ZTR21_CLASS**)，扮演「合法值清單」的角色
- **外鍵表 (Foreign Key Table)**：帶外鍵欄位的表 (本例 **ZTR21_STUD** 的 **KLASSE** 欄位)，欄位值必須存在於檢查表裡
- 一對多：一個班級 (Header) 可以有多個學生 (Detail)，基數 (Cardinality) 設 **1 : CN** (說明見下方)

SE11 用 Dictionary DDL (新版原始碼式編輯) 寫的話，語法長這樣 (跟講義 6 的 **SPFLI** 外鍵到 **SCARR** 是同一套)：

```
define table ztr21_stud {  
  ...  
  @AbapCatalog.foreignKey.label : 'Check Against Class'  
  @AbapCatalog.foreignKey.screenCheck : true  
  klasse : ztr21_klasse  
    with foreign key [0..*,1] ztr21_class  
      where mandt = ztr21_stud.mandt  
        and klasse = ztr21_stud.klasse;  
  ...  
}
```

[0..*,1]：左邊 **0..*** 是外鍵表這一側 (一個班級可以對應任意多個學生，也可以一個都沒有)、右邊 **1** 是檢查表那一側 (每個學生的 **KLASSE** 在 **ZTR21_CLASS** 剛好對應 1 筆)。

基數 (Cardinality) : 兩張表的資料是幾對幾

設定外鍵時要填基數，描述「外鍵表的資料」和「檢查表的資料」之間是幾對幾。SE11 畫面上寫成 **n : m** 兩格 (官方文件 [ABENDDIC_DATABASE_TABLES_FORKEY](#)) :

位置	回答的問題	可選的值
左邊 n (檢查表這一側)	外鍵表的 每一筆 ，在檢查表裡有幾筆對應？	1 : 剛好一筆，一定要有 C : 最多一筆，也可以沒有
右邊 m (外鍵表這一側)	檢查表的 每一筆 ，在外鍵表裡有幾筆對應？	1 : 剛好一筆 C : 最多一筆，也可以沒有 N : 至少一筆 CN : 任意筆數，可以沒有

怎麼決定：拿兩個問題問自己，照答案選。以本課的學生表 (外鍵表) 與班級表 (檢查表) 為例：

1. 每個學生屬於幾個班級？剛好一個 → 左邊填 **1**
2. 每個班級有幾個學生？可以很多、剛成立的班級也可能還沒有學生 → 右邊填 **CN**

所以是 **1 : CN**。本課程幾張表在系統裡的設定 (2026-09-30 查 [DD08L](#)) :

外鍵表 → 檢查表	基數	意思
ZTR21_STUD → ZTR21_CLASS	1 : CN	每個學生一個班級；每個班級任意多個學生
ZTR23_ORDI → ZTR23_ORDH (講義 23)	1 : CN	每筆明細屬於一張訂單；每張訂單任意多筆明細
ZTR25_SURCHG → SCARR (講義 25)	1 : C	每筆加成設定對應一家航空公司；每家航空公司 最多一筆 加成設定

DDL 的寫法順序剛好相反：**with foreign key** [外鍵表這一側，檢查表這一側]，數量用範圍表示：

SE11	DDL	說明
1 : CN	[0..*,1]	0..* 對應 CN 、 1 對應 1
1 : C	[0..1,1]	0..1 對應 C
1 : N	[1..*,1]	1..* 對應 N

基數有什麼作用：

- 主要是文件用途，讓看表的人知道兩張表的關係。外鍵的畫面檢查 (Screen Check) 不會因為基數不同而改變行為。
- 建 **Maintenance View** 或 **Help View** 時，系統會看基數決定哪些表可以加進 View (講義 21a)。基數填錯，可能加不進去，或維護畫面行為不符合預期。
- 所以照實際的資料關係填就好，不要隨便選。

外鍵欄位類型 (**Foreign key field type**) : 外鍵欄位在自己這張表裡扮演什麼角色

SE11 外鍵對話框的 **Semantic attributes** 區塊，除了基數還有一組選項，描述「外鍵欄位在外鍵表裡是不是主鍵」（官方文件 [ABENDDIC_DATABASE_TABLES_FORKEY](#)）：

選項	意思	什麼時候選	系統範例 (2026-09-30 查 DD08L)
Not Specified	不說明	不想交代時的預設	本課程 ZTR21 / ZTR23 / ZTR25 / ZTR28 的外鍵都沒設定
Non-key fields/candidates	外鍵欄位 不是 外鍵表的主鍵，也不能唯一識別一筆資料	外鍵欄位只是一般資料欄位	學生表的 KLASSE (班級代碼不是學生表的主鍵)
Key fields/candidates	外鍵欄位 是 外鍵表主鍵的一部分 (或能唯一識別一筆資料)	外鍵欄位本身就是 Key	MARC-WERKS (工廠是物料工廠資料的主鍵之一) ； 訂單明細的 ORDNO (講義 23)
Key fields of a text table	外鍵表是檢查表的 文字表	建「代碼 + 各語言說明」的文字表時	MAKT-MATNR (物料說明， MARA 的文字表，見下方) ； T005T-LAND1 (國家名稱， T005 的文字表)

- 前兩種 (Non-key、Key) 只是文件用途，不影響系統行為，讓看表的人知道這個外鍵欄位的角色。
- **Key fields of a text table** 會真的改變系統行為：系統把這張表當成檢查表的文字表，F4 選單會自動帶出說明文字，Maintenance View 也能自動只取登入語言的那一筆 (講義 21a)。條件是：外鍵表的主鍵要跟檢查表一樣，再多一個 **LANG** 型別的語言欄位；每張檢查表只能有一張文字表。
- 本課程的練習照實際關係選即可，例如學生表的 **KLASSE** 選 Non-key fields/candidates；選 Not Specified 也不影響功能。

文字表 (Text Table) 的實例：MARA 與 MAKT

物料主檔 **MARA** 一個物料一筆，但物料說明要能用不同語言顯示：中文登入看到中文說明、英文登入看到英文說明。說明如果放在 **MARA** 裡，一個欄位只放得下一種語言，所以 SAP 另外建一張 **MAKT** 專放說明，每個物料、每種語言各一筆：

表	主鍵	其他欄位	資料長相
MARA (檢查表)	MANDT 、 MATNR	物料類型、基本單位.....	物料 A001 一筆
MAKT (文字表)	MANDT 、 MATNR 、 SPRAS (語言， LANG 型別)	MAKTX (物料說明)	A001 + 中文一筆、 A001 + 英文一筆.....

MAKT-MATNR 的外鍵指向 **MARA**，Foreign key field type 選 **Key fields of a text table**，基數 1：CN (每筆說明屬於一個物料；每個物料可以有任意多種語言的說明)。

設成文字表之後，系統會：

- 在 **MATNR** 欄位按 F4 時，選單自動帶出登入語言的物料說明。
- 在 Maintenance View (講義 21a) 裡，自動只取登入語言的那一筆說明來顯示和維護。

程式讀文字表，一定要加語言條件：

```
SELECT SINGLE maktx FROM maktx INTO gv_maktx
WHERE matnr = gv_matnr
AND spras = sy-langu.      " 只取登入語言那一筆
```

- 少了 `spras = sy-langu`，同一個物料會讀到好幾筆（每種語言一筆），`SELECT SINGLE` 讀到哪一種語言不一定。
- 說明可能還沒翻成登入語言，查不到時 `sy-subrc = 4`，程式要處理（例如顯示空白）。講義 11 學 `JOIN` 之後，會看到實際報表 `Z_INVENTORY_COST_REPORT` 用 `LEFT OUTER JOIN maktx ... AND spras = sy-langu`：用 `LEFT OUTER` 是為了說明不存在時，物料照樣列出來。

自己建文字表的規則：主鍵跟檢查表相同，再多一個語言欄位（引用標準 Data Element `SPRAS`）；每張檢查表只能有一張文字表。

3.2 外鍵只在「畫面輸入」擋人，不是資料庫 constraint

這是本題最容易搞錯的地方：`@AbapCatalog.foreignKey.screenCheck : true` 只影響畫面（SM30、Module Pool 的輸入欄位）在使用者離開欄位時做檢查——**Open SQL 的 `INSERT/UPDATE/MODIFY` 完全不會被擋**，程式塞一個 `ZTR21_CLASS` 沒有的班級代碼一樣會成功（`sy-subrc = 0`）。想在程式層也擋，要自己 `SELECT SINGLE` 檢查檢查表存不存在該筆，這不是系統自動做的事。

（跟其他資料庫的「Foreign Key Constraint」不一樣：那是資料庫引擎強制擋寫入；SAP DDIC 外鍵是應用層 / 畫面層的檢核機制，這個落差是很多人踩過的坑。）

本講兩邊都會實測：第 4 節在 SM30 輸入不存在的班級會被擋；第 5 節的練習程式用 Open SQL 寫入同樣的值卻會成功。

3.3 Search Help：F4 選單哪裡來的

Search Help（搜尋輔助）解決的是另一個問題：**使用者不用背代碼，可以用 F4 選**。跟外鍵是兩件事——外鍵負責「擋不合法的值」，Search Help 負責「幫你選合法的值」，兩者可以疊加也可以只設一個：

- 只設外鍵沒設 Search Help：畫面會擋錯誤輸入，但沒有 F4 選單，要自己背代碼
- 只設 Search Help 沒設外鍵：F4 能選，但使用者若不透過 F4、手動打一個不存在的代碼，畫面不會擋

建立方式（SE11 → Search Help → Elementary Search Help）：

- **Selection Method**：資料來源表（本例 `ZTR21_CLASS`）
- **Search Help Parameters**：每個要出現在選單裡的欄位；哪一個是「查完之後要帶回畫面」的欄位（本例 `KLASSE`）勾 Import+Export+SH field，純顯示用的欄位（`KLNAME`）只勾 Export
- 建好後要掛到 **Data Element**（`ZTR21_KLASSE` 的 Further Characteristics 頁籤填 Search Help 名稱）才會在所有用到這個 DE 的欄位自動生效——這也是三層件「改一處全生效」精神的延伸

除了在 SM30 按 F4（第 4 節），也可以寫一支只有選擇畫面的小程式，參數參考這個 Data Element（講義 7）來測：

```
PARAMETERS p_klasse TYPE ztr21_klasse.      " F4 會帶出 ZTR21_CLASSH 的班級清單
```

實測踩過的坑：Selection Method 表（`ZTR21_CLASS`）裡每個要當 Search Help Parameter 的欄位，都必須引用一個 **Data Element**——`KLNAME` 一開始貪方便直接用內建型別 `CHAR(40)`，結果 Search Help

Activate 失敗。Search Help 靠 Data Element 才能解析欄位的語意（標籤、型別），純內建型別的欄位沒有這層資訊可以掛。這也是三層件「共用 Data Element」精神的另一個實際理由：不是只有標籤好看，Search Help 這種進階功能還真的依賴它。

Search Help 的資料來源除了一張表，也可以是 Database View 或 Help View——這兩種要先學 JOIN（講義 11），放在講義 21a 介紹。

4. SM30 維護畫面

讓使用者（或顧問）不寫程式就能維護表內容：

1. 先建 **Function Group**：SE80 → 下拉選 Function Group → 輸入 **ZFG_TR21** → Yes → 填 Short Text → 存檔（Function Group 是產生出來的維護畫面程式的容器，講義 25 的練習已經建過 **ZFG_TR25**，完整觀念在講義 15；Table Maintenance Generator 只能放進已經存在的 Function Group）
2. SE11 該表 → Utilities → **Table Maintenance Generator**
3. Authorization Group 練習用 **&NC&**（不檢核）；Function Group 填 **ZFG_TR21**；Maintenance type 選 one step（單畫面）
4. 產生後，SM30 輸入表名 → Maintain，就有現成的新增/修改/刪除畫面

前提：建表時 Data Browser/Table View Maint. 要選 **Display/Maintenance Allowed**（第 2 節），否則 SM30 會說這張表不能維護。

實務上參數表、對照表幾乎都配 SM30；正式環境的維護權限與是否產 TR 由 Delivery Class 與權限控制（進階的權限防護見講義 28）。

4.1 在 SM30 驗證 DDIC 設定

前面在 SE11 設好的東西，到了畫面上才看得到效果：

SE11 的設定	在 SM30 的效果
Domain ZTR21_SCORE 值域 0 ~ 999	成績輸入 1000 會被擋下
Data Element 的欄位標籤	欄位標題顯示「學生成績」「班級代碼」
KLASSE 外鍵 + Screen Check	輸入 ZTR21_CLASS 沒有的班級代碼，離開欄位時報錯
Search Help 掛在 ZTR21_KLASSE	班級欄位按 F4 帶出班級清單

對照本講的練習程式（第 5 節的 Open SQL）：同樣是「班級不存在」，程式用 Open SQL **INSERT** 會成功（**sy-subrc = 0**），SM30 畫面輸入卻會被擋。這就是「外鍵只擋畫面、不擋 Open SQL」的實證。

5. Open SQL 寫入

讀是 SELECT；寫有四個指令，全部用 **sy-subrc** 回報結果（鐵律不變），**sy-dbcnt** 是影響筆數：

```
DATA gs_stud TYPE ztr21_stud. " 直接用表名當結構型別
```

```
* INSERT：新增一筆；主鍵已存在 → sy-subrc = 4，不會蓋掉
gs_stud-id      = 'S0001'.
gs_stud-name    = '王小明'.
```

```

gs_stud-score = 85.
INSERT ztr21_stud FROM gs_stud.

* UPDATE：改既有資料；找不到 → sy-subrc = 4
UPDATE ztr21_stud SET score = 90 WHERE id = 'S0001'.

* MODIFY：有就改、沒有就新增 ( upsert ) — 參數表最愛用
MODIFY ztr21_stud FROM gs_stud.

* DELETE：刪除
DELETE FROM ztr21_stud WHERE id = 'S0001'.

```

多筆版本：**INSERT ztr21_stud FROM TABLE gt_stud.** (UPDATE/MODIFY/DELETE 同理有 **FROM TABLE**) 。
 注意 INSERT FROM TABLE 遇到任一筆主鍵重複就整批 **dump**，除非加 **ACCEPTING DUPLICATE KEYS** (重複的跳過、sy-subrc = 4) 。

- MANDT 欄位一樣不用 (不要) 自己塞，系統自動帶當前 client。
- 寫入欄位建議程式帶齊稽核欄 (如異動者 **sy-uname**、異動日 **sy-datum**) ——查問題時會感謝自己。

6. LUW 與 COMMIT WORK

資料庫的變更不是逐句永久生效，而是以 **LUW (Logical Unit of Work)** 為單位，最後一次「確認」才真正落地：

```

INSERT ztr21_stud FROM gs_stud.
IF sy-subrc <> 0.
    ROLLBACK WORK.           " 整包撤銷：這個 LUW 內所有寫入取消
    WRITE / '寫入失敗，已回復'. " 正式程式用 MESSAGE 通知使用者 ( 講義 10、22 )
    RETURN.
ENDIF.
COMMIT WORK.                " 整包確認：全部永久生效

```

- **概念**：一個業務動作的多筆寫入 (如表頭 + 明細) 必須「全成功或全失敗」，不能寫一半——這就是 LUW 的意義。
- 報表程式跑完 (或畫面切換) 時系統會隱含 **commit**——所以練習程式沒寫 COMMIT WORK 資料多半也進去了，但正式程式的寫入要明確 **COMMIT / ROLLBACK**，把「哪裡算一個完整動作」寫清楚。
- 多人同時改同一筆怎麼辦？正式做法要配 **Lock Object** (SE11 建 **EZ...**，產生 ENQUEUE/DEQUEUE FM) ——本課先認識名詞，實作在講義 27。

7. 常見錯誤與陷阱

症狀	原因
表啟用不了	Technical Settings 沒填 (Data Class / Size Category)
啟用報 specify reference table AND reference field	金額 (CURR) 或數量 (QUAN) 欄位沒指定幣別 / 單位參考欄位 (見 2.2)

症狀	原因
SE16N 看不到剛寫的資料	寫入時 sy-subrc 其實是 4 (沒檢查) ; 或在別的 client 查
INSERT 一直 subrc = 4	主鍵重複——練習程式重跑前先 DELETE 舊測試資料，或改用 MODIFY
INSERT FROM TABLE 直接 dump	批次裡有主鍵重複，沒加 ACCEPTING DUPLICATE KEYS
自己塞 MANDT	不用，系統自動處理 (跟 SELECT 一樣)
以為外鍵能擋住程式寫入的髒資料	外鍵 (screenCheck) 只管畫面輸入，Open SQL 不受影響 (見 3.2)
SM30 說表不能維護	建表時 Data Browser/Table View Maint. 選了不允許，或沒產 Maintenance 畫面
Table Maintenance Generator 產生失敗	Function Group 還沒建，或建在別的套件
SM30 輸入不存在的班級沒被擋	外鍵沒打開 Screen Check，或表改完沒重新啟用
DE 掛了 Search Help 但 F4 還是沒選單	Search Help 本身沒啟用，或掛的是舊版本；兩邊都要各自啟用一次
Search Help 存檔/Activate 失敗	Selection Method 表裡的欄位 (如 KLNAME) 沒引用 Data Element，只用了內建型別

8. 課堂練習

完成 ex21：建 Domain + Data Element + 透明表 ZTR21_STUD；再建 Header 表 ZTR21_CLASS、幫 KLASSE 欄位設外鍵與 Search Help；產生 SM30 維護畫面，在畫面上驗證值域、外鍵、F4；最後寫程式跑完 INSERT / UPDATE / MODIFY / DELETE 全流程 + 外鍵行為驗證，並驗證 sy-subrc。